Vietnam National University Ho Chi Minh City
Ho Chi Minh City University of Technology
Faculty of Computer Science and Engineering

– INTERNSHIP 2 PROPOSAL –

Cost-Aware Scheduling of Test Evidence for LLM-Based Program Repair

Council: Computer Science and Engineering
Instructor: Dr. Le Xuan Bach, CSE-HCMUT
Student: Nguyen Thanh Trung Dat – 0000000

Ho Chi Minh City, November 2026

Date: _____

Dr. Le Xuan Bach, CSE-HCMUT (Internship Instructor)
Faculty of Computer Science and Engineering

Acknowledgements

I would like to thank the lecturers of the Faculty of Computer Science and Engineering, Ho Chi Minh City University of Technology (HCMUT), for the foundational and specialised teaching that made this work possible.

I am especially grateful to my instructor, Dr. Le Xuan Bach, CSE-HCMUT, for the guidance that shaped the scope of this proposal. Two pieces of that guidance did most of the work. The first was the insistence that a research question be stated together with the outcome that would *refute* it; applying that test removed three questions from an earlier roster that turned out to be names for groups of experiments rather than questions. The second was the instruction to measure the resource the system actually spends rather than the one the literature happens to report, which is the reason the preliminary study in Chapter 4 reports CPU seconds and test executions side by side, and the reason it found that the two disagree in sign.

This proposal also stands on a previous study of my own, CegMem, in which a counterexample-guided repair loop stored every refuted patch under a failure type keyed by its edit location. That study answered its question and, in answering it, exposed the one this proposal is about: a memory that knows *which patches not to try again* still says nothing about *which test to run first*. I thank my colleagues for the discussions that made that distinction sharp enough to build on.

This document reports work in progress. The preliminary validation in Section 4.3 is a replay and re-execution study on the frozen data of the earlier project; the full grid on the Caster implementation — the twenty-two arms, the steered prompt contract, and the complete fault corpus — has not been run, and is stated as Phase 2 work. Any remaining errors are my own.

Abstract

A large-language-model program-repair loop alternates between a proposer, which draws a candidate patch, and an oracle, which refutes it by finding a test case on which the patch is wrong. The proposer is the half that gets the attention and the oracle is the half that costs. On the corpus studied here the oracle's own reference program takes a median of 0.076 s of CPU per test case but a 99th percentile of 4.48 s — a spread of 58.7× inside the same pool — and a loop that walks that pool in the order the benchmark ships spends 2.18 s of CPU per round, on every round, for twenty rounds.

Existing memories for repair loops do not touch that cost. They are keyed by a failure *class*: fault-recorded testing prioritisation ranks a case by how many patches it has already falsified, and the predecessor of this work, CegMem, stored each refuted patch under a type formed from its edit location and the property it violated. A class-keyed memory can say which patches not to try again — CegMem cut oracle calls per episode from 11.11 to 1.57, a factor of 7.07 — but it cannot say which test to run first, and it cannot say which test is cheap.

This proposal develops Caster, a repair loop whose memory is keyed by **test case** rather than by failure class. Each case carries an online Beta-posterior kill rate K and an online mean CPU cost $\hat{m}$, and probes are scheduled by the index $K/\hat{m}$: refute early, and refute cheaply. Keying on the case is also what makes the second half possible, because a case is an *observation* and only an observation can be phrased to the proposer as a positive requirement rather than a prohibition.

What this document reports as evidence. The preliminary validation is a pilot on the frozen data of the CegMem study: 68 ConDefects tasks, 2,523 test cases and 4,670 stored patches for which a complete patch × case verdict matrix, a measured per-case reference CPU cost, and a per-case coverage trace all exist. Two experiments run on it. A replay simulation reproduces an arbitrary probe order exactly from the verdict matrix; a second study re-executes the loop for real over 6,800 rounds per policy and measures CPU with `getrusage`. The measured result is that $K/\hat{m}$ costs **76.2 %** less oracle CPU than the shipped pool order (95 % CI $[-84.9, -61.2]$, $p = 1.7 \times 10^{-11}$, cheaper on 63 of 68 tasks) and **15.7 %** less than the published fault-recorded baseline (95 % CI $[-28.9, -5.1]$, $p = 0.011$, 42 of 68) — while paying 23.3 % *more* probes than that baseline. Executions and CPU therefore disagree in sign on this corpus, which

matters because the test-prioritisation literature deliberately reports executions.

What it does not report. The CASTER implementation itself has been built and unit-tested but its experimental grid has not been run. The twenty-two arms, the steered prompt contract that the test-case key is supposed to enable, the duplicate-rate and token-cost measurements, and the full fault corpus are Phase 2 work and are stated as such throughout. No number in Chapter 5 is a result; every number there is a target.

Contents

Acknowledgements **iv**

Abstract **v**

1 Introduction **1**
1.1 Problem Statement 1
1.2 Research Objectives and Questions 2
1.3 Scope 4
1.4 Report Structure 6

2 Related Works **7**
2.1 Survey of Representative Works 7
2.2 Comparison Table 9
2.3 Research Gap 9
2.4 Direct Competitors 11

3 Theoretical Background **12**
3.1 The Repair Loop and Its Two Costs 12
3.2 Test-Case Prioritisation 13
3.3 Fault-Recorded Prioritisation and What It Discards 13
3.4 The Premise: CegMem's Typed Memory 14
3.5 The $K/\hat{m}$ Index 15
3.6 Cost Accounting 16
3.7 Benchmark Infrastructure 16

4 Proposed Framework **18**
4.1 System Overview 18
4.1.1 System Requirements 19
4.2 Module Implementation Details 19
4.2.1 The Ledger 19
4.2.2 The Oracle and the Stores 20
4.2.3 The Steering Contract 20
4.3 Preliminary Validation 20
4.3.1 Why the Pilot Runs on the Predecessor's Data 20

4.3.2 The Arena . . . 21
4.3.3 Measured Head-to-Head . . . 22
4.3.4 Replay Head-to-Head . . . 25
4.3.5 How Much to Divide by Cost . . . 26
4.3.6 Soundness Check . . . 26
4.3.7 Reproducibility . . . 27
4.4 Discussion . . . 27

5 Evaluation Plan **28**
5.1 Metrics . . . 28
5.2 The Arm Roster . . . 29
5.3 Ablation Ladder . . . 30
5.4 Sensitivity . . . 30
5.5 Protocol and Tooling . . . 31
5.6 Baseline Comparisons . . . 33
5.7 Threats to Validity . . . 34

6 Summary and Future Plan **35**
6.1 Summary . . . 35
6.2 Future Direction . . . 36
6.3 Limitations . . . 38
6.4 Timeline . . . 38

References **41**

List of Figures

4.1 One round of the CASTER loop. Stage 2 is the contribution of RQ1–2; stage 4 is the contribution of RQ3–4 and is not exercised by the pilot. . 18
4.2 Measured oracle cost and probe count on the arena. The two panels rank the policies in *opposite* orders. 23
4.3 Measured oracle CPU per round. The incumbent never learns; both memories collapse within about three rounds; $K/\hat{m}$ continues to separate from FRTP as its cost model fills in. 24
4.4 Per-task measured CPU, $K/\hat{m}$ against FRTP. The win is broad rather than driven by outliers, but it is not universal. 24
4.5 Cost-exponent sweep, 20 seeds, paired per task. The optimum is interior: dividing by cost helps, dividing by the *full* cost over-corrects. 26

6.1 Project timeline. Everything up to and including the proposal-defence milestone has been carried out; every bar below it is planned work. Deliverables per work package are in Table 6.2. 39

List of Tables

1.1 Project scope. 5

2.1 Representative works against the four dimensions. "—" means the dimension is absent from the system. 9
2.2 Direct competitors and the differentiator in each case. 11

3.1 CegMem main grid: 309 tasks, 5 seeds, budget 20, 1,545 paired cells per arm, `gpt-4o-mini`. Per-episode means. 14

4.1 Per-round data flow. 18
4.2 The pilot arena, and the two distributions that motivate the design. . . 21
4.3 Measured head-to-head on the arena. 6,800 rounds per policy, CPU from `getrusage`. "cheaper on" counts tasks where the row is cheaper than $K/\hat{m}$. 22
4.4 Paired contrasts, cluster bootstrap over 68 tasks. 22
4.5 Replay over the arena, 20 seeds. Δ is against plain $K/\hat{m}$; CIs are a cluster bootstrap over the 68 tasks. 25

5.1 Oracle-side metrics (RQ1, RQ2). 28
5.2 Proposer-side metrics (RQ3, RQ4). 29
5.3 The ablation ladder. One variable per step. 30
5.4 Tools and their role. 31
5.5 Tier A — reproduced head-to-head on our corpus. 33
5.6 Tier B — cited qualitatively from the original papers; not reproducible within this project's budget. 34

6.1 Limitations and planned mitigations. 38
6.2 Work packages by phase. 40

Chapter 1

Introduction

1.1 Problem Statement

Automated program repair (APR) has converged on a loop with two halves. A *proposer* produces a candidate patch, and an *oracle* decides whether that candidate is wrong by running it against a pool of test cases and looking for one on which it misbehaves[1]. In the generate-and-validate tradition the proposer was a mutation operator[2]; today it is usually a large language model[3]. The oracle has not changed: it is still a sequence of program executions, and it is still the half that burns the machine.

How much it burns is easy to under-estimate, because the usual unit of account hides it. The test-prioritisation literature reports *test executions* rather than time, and does so deliberately: the largest recent study of patch validation states that it presents results in executions because "the number of test executions is more stable and more suitable for future work to reproduce"[4]. That choice is sound when executions are interchangeable. On the corpus studied in this proposal they are not. The oracle's own reference program takes a median of 0.076 s of CPU on one test case and 4.48 s at the 99th percentile — a 58.7× spread — and the variation is not between easy and hard *tasks* but inside a single task's pool, where the median coefficient of variation of per-case cost is 0.571. Counting executions on such a pool is counting apples and buses.

The consequence shows up directly in the loop. A repair loop that walks the pool in the order the benchmark happens to ship — which is what any implementation gets for free, and therefore what most implementations do — spends 2.18 s of oracle CPU per round and keeps spending it: the cost per round is flat from round 1 to round 20, because nothing in the loop learns anything about the pool. Over the 6,800 rounds measured in Section 4.3 that order costs 14843 s of CPU where a scheduled order costs 3532 s.

Two kinds of memory have been proposed to make the loop cheaper, and neither addresses this. *Fault-recorded testing prioritisation* (FRTP) ranks a test case by the number of patches it has already falsified[5]; the same quantity, under the name "patch

killing count", drives later variants[6]. CegMem, the study that precedes this one, went further and stored every refuted patch under a *failure type* formed from the patch's edit location and the property it violated, so that a fresh candidate matching a stored type could be rejected before the oracle was paid at all. That worked: it cut oracle calls per episode from 11.11 to 1.57, a factor of 7.07 (Section 3.4).

But both memories are keyed by a *class*, and a class-keyed memory answers only one of the two questions a repair loop asks. It can answer *which patches are not worth testing*. It cannot answer *which test to run first*, and it certainly cannot answer *which test is cheap* — Frtp's whole family records that a case killed, never what the kill cost, and never the evidence of a case failing to kill, which is the information that distinguishes a case that spared ten patches from one that was never run. In CegMem the residual was plainly visible: the memory removed most oracle *calls*, yet the loop still performed 39.5 sandbox executions per episode and still spent 14.1 s per episode inside the guard, because every surviving call still walked the pool in alphabetical order.

This proposal targets that residual. The idea is to key the memory by **test case** instead of by failure class, to learn two online quantities per case — a kill rate and a cost — and to schedule probes by their ratio. Keying on the case has a second consequence that motivates the rest of the design: a test case is an *observation*, and only an observation can be handed back to the proposer as a positive requirement ("produce exactly this output on this input") rather than as a prohibition ("do not do that again"). The scheduler and the prompt contract are therefore one system, and they ship together.

1.2 Research Objectives and Questions

This research aims to design, implement and evaluate a repair loop whose memory of refuted attempts is indexed by test case, and to establish how much of the oracle's cost that indexing removes. Four objectives structure the work; each is paired with the question it answers and, following the discipline adopted throughout this project, with the outcome that would *refute* it. A question with no refuting outcome is a name for a group of experiments, not a question.

1. **Cost-aware probe scheduling.** Schedule the oracle's probes by an online per-case kill rate divided by an online per-case cost, rather than by pool order or by a kill count. *Research question (RQ1).* Does scheduling by $K/\hat{m}$ reduce the CPU spent refuting a patch, measured against the incumbent pool order, against an uninformed random control, and against the published prioritisers Frtp and additional-greedy

coverage? *Refuted when* $K/\hat{m}$ is not cheaper than pool order beyond the cluster bootstrap's interval; or when it is cheaper than pool order but not cheaper than FRTP and coverage, in which case the contribution is early stopping, which is not ours. *Status.* Answered in the affirmative by the pilot of Section 4.3; to be re-established on the full corpus in Phase 2.

2. **Attribution of the effect.** Decompose the index into the three things it adds to the incumbent — having a memory at all, turning a count into a rate, and dividing that rate by cost. *Research question (RQ2).* Which of the three terms produces the effect? *Refuted when* no single step clears the bootstrap interval, so the gain is "having any memory" and the score is ornament; or when the cost division alone carries it, in which case the kill rate is ornament. *Status.* Partially answered by the pilot's exponent sweep (Section 4.3.5), which separates the kill term from the cost term; the full stepwise ladder is Phase 2.

3. **Renormalising the patch family.** Return the scheduled counterexample to the proposer as a positive requirement keyed to a test case. *Research question (RQ3).* Does doing so move the proposal distribution off the mode it has already drawn — fewer duplicate patches, more repairs at the same budget — where a conversation transcript and a prohibition do not? *Refuted when* the duplicate rate does not fall against independent repeated sampling; or when it falls and the repair rate does not rise, in which case the distribution moved somewhere no better. *Status.* Not addressed by the pilot. The baseline is set by published measurement: independent repeated prompting has already been reported to repair 23 bugs against conversational repair's 18 at an equal budget of 24 model calls, with 65 % of the conversational system's patches duplicates[7]. Any steering arm must beat repeated sampling, not the pool order.

4. **The disclosure dose.** Showing a counterexample tells the model what to satisfy. *Research question (RQ4).* At what level of disclosure does the proposer begin satisfying *the shown probes* rather than repairing the fault? *Refuted when* the share of patches refuted only outside the disclosed set is flat across the number of facts shown and across disclosure level, in which case there is no dose and the axis does not exist. *Status.* Phase 2.

Contribution positioning. None of the individual ingredients is new. Cost-cognizant prioritisation dates to Elbaum et al.[8]; the additional-greedy family to Rothermel et al.[9]; prioritisation inside a repair loop to Qi et al.[5]; and Beta-posterior estimates with ε-greedy exploration are standard. The claim made here is at the level of the *key*

and the unit. First, the memory is indexed by test case, which is what lets one record serve both the scheduler and the prompt; every memory in the surveyed repair literature is indexed by a class. Second, the ranked quantity is a rate divided by a measured cost, where the published repair-loop prioritiser ranks on a count and discards both the non-kills and the cost. Third — and this is the finding the pilot contributes rather than the design — on a pool with 58.7× internal cost spread the two units of account disagree in sign, so the deployment-facing claim cannot be read off an execution count at all.

1.3 Scope

The study is bounded to the dimensions in Table 1.1.

Table 1.1: Project scope.

Dimension	**Specification**
Task domain	Single-fault repair of competitive-programming submissions; one faulty program, one contest test pool, stdout-compared verdicts.
Benchmark	ConDefects[10], Python subset. Chosen because every test input ships with the output the judge accepted, so a refutation is a concrete input/output pair rather than a pass/fail bit.
Proposer	One OpenAI-compatible chat model per grid; `gpt-4o-mini` for the data the pilot reuses. The proposer is not a variable of the study.
Oracle	One program, one input, hard timeout, CPU measured with `getrusage(RUSAGE_CHILDREN)`; probing stops at the first refutation.
Cost unit	CPU seconds as primary, test executions reported alongside. Wall-clock time is never used as a cost, because it is contaminated by sibling processes.
Core method	Per-case Beta-posterior kill rate K over an online per-case mean cost $\hat{m}$, ranked by $K/\hat{m}$, with ε-greedy exploration.
Evidence in this document	A replay and re-execution pilot on the frozen CegMem data: 68 tasks, 2,523 cases, 4,670 patches, 6,800 measured rounds per policy.
Out of scope here	The Caster grid itself (22 arms), the steered prompt contract, duplicate-rate and token measurements, multi-fault and multi-language repair, and any claim about repair *quality* beyond the soundness invariant of Section 4.3.6.

Scope of completed work at the proposal stage. The Caster implementation exists: ten command-line verbs, the module boundaries of Section 4.1, and 263 unit tests that run without a network, a model or the benchmark. What has *not* happened is the experiment. No arm of the grid has been run to completion on the full corpus, no steered arm has been run at all, and therefore nothing in this document derived from the Caster repository is reported as a result. The evidence is the pilot, and the pilot runs on the previous project's data by design — see Section 4.3.1 for why that is the stronger choice and not merely the available one.

1.4 Report Structure

Chapter 2 surveys validation-cost work along four families — generate-and-validate repair and its validation bill, regression test prioritisation, repair-specific prioritisation, and the token economics of LLM repair loops — consolidates them into one comparison table, and states the gaps. Chapter 3 gives the theory: the two costs of the repair loop, total versus additional greedy prioritisation, what FRTP records and what it throws away, the typed memory of CEGMEM that this work starts from, the Beta-posterior estimator behind K, and the cost-accounting rules. Chapter 4 describes the proposed system and reports the preliminary validation in full. Chapter 5 is the evaluation plan: metrics, the arm roster, the ablation ladder, and the two tiers of baseline. Chapter 6 summarises, lists limitations honestly, and gives the phase timeline.

Chapter 2

Related Works

Work relevant to this proposal falls into four families: (1) generate-and-validate repair and the size of its validation bill, (2) regression test-case prioritisation, which is where the scheduling machinery comes from, (3) prioritisation specialised to repair loops, which is where the direct baseline lives, and (4) LLM-based repair loops and their token economics, which is where the proposer-side half of the design belongs. This chapter walks the four, consolidates them into one table, and distils the gaps.

2.1 Survey of Representative Works

Generate-and-validate repair and the validation bill. GenProg[2] established the loop that the rest of this literature refines: mutate, validate against the test suite, keep what survives. Its systematic study repaired 55 of 105 real bugs[11], and validation dominates its runtime, because each candidate is run against the suite from scratch. The structural weakness of validating against a test suite was made precise by Qi et al.[12], who re-examined the patches reported by three generate-and-validate systems and found that most were *plausible* rather than correct — GenProg produced correct patches for 2 defects out of 18 for which it produced a plausible patch, AE 3 of 27, and RSRepair 2 of 10. That result is the reason this proposal treats the strength of the oracle (how many probes a patch must survive) as a parameter to be stated rather than an implementation detail, and the reason the soundness invariant of Section 4.3.6 is asserted on every cell. Defects4J[13] supplied the benchmark the family standardised on; ConDefects[10] supplies the one used here, for the data-leakage reason given in Section 1.3.

Regression test-case prioritisation. Rothermel et al.[9] defined the problem and the two greedy families that still dominate it: *total* strategies, which rank a test by how much it covers, and *additional* strategies, which rank by how much it covers that nothing scheduled so far has covered. The survey of Lou et al.[14] organises two decades of refinements of those two. Crucially for this proposal, Elbaum et al.[8] observed early

that the field's standard metric assumed uniform test costs and uniform fault severities, and proposed a cost-cognizant replacement. That paper is the philosophical ancestor of the $\hat{m}$ term here; what it did not have, and what this work adds, is an *online* cost estimate learned inside the loop rather than a cost supplied in advance.

Prioritisation inside a repair loop. Qi et al.[5] introduced fault-recorded testing prioritisation (FRTP), implemented as TRPAUTOREPAIR, and evaluated it against GENPROG on five subject programs with sixteen real bugs: it matched or beat GENPROG's success rate and improved repair efficiency in fifteen of the sixteen. FRTP ranks a test case by its *failing count* — the number of validated patches the case has already falsified — and gathers that information from the repair process itself rather than from a separate profiling pass. It is the published baseline for this work, and the arm named `fault-record` in Chapter 5 implements exactly that ranking, not a weakened variant. Venugopal et al.[6] extend the idea with modification-point awareness, maintaining a table whose entries are the "patch killing count" of a test case against a program variant, which is the same quantity under another name.

The most thorough measurement of patch-validation cost is Lou et al.[4], who study 2,532,915 patches produced by twelve APR systems on Defects4J. Two of their findings bear directly on this proposal. First, they report results in test executions rather than seconds, stating that executions are "more stable and more suitable for future work to reproduce". Second, they find that *selection* dominates *prioritisation* in their setting: class-level regression test selection alone reduces executions by about 37 % for PRAPR, where the best prioritisation approach reduces them by only about 15 %. Both findings are properties of their setting rather than of the problem, and Section 2.3 explains why neither transfers to the one studied here.

LLM repair loops and their token economics. CHATREPAIR[3] made the conversational loop standard, repairing 162 of 337 Defects4J bugs at roughly $0.42 each by feeding failing-test feedback back into the conversation. CIGAR[15] attacked its cost directly and reports 127K tokens per bug against a reimplemented CHATREPAIR baseline's 467K, a 73 % reduction, while producing correct patches for 171 of 429 bugs. The sharpest result in the family is negative: Chen et al.[7] hold the budget fixed at 24 model calls per bug and find that *independent repeated prompting* repairs 23 bugs where CHATREPAIR repairs 18, and that 65 % of CHATREPAIR's generated patches are duplicates. The obvious iterative strategy is therefore already published as worse than simply drawing again, which fixes the baseline any steering arm in this project must beat.

The immediate predecessor. CegMem — the author's previous study, described in detail in Section 3.4 — sits between the two halves. It keyed a memory of refuted patches by a failure type θ = (edit location) × (violated property) and used it both to block redundant candidates before the oracle ran and to steer the proposer away from exhausted classes. On 309 ConDefects tasks it cut oracle calls per episode by 7.07× and raised the repair rate from 0.526 to 0.581. It is cited here as the premise of this proposal rather than as a competitor.

2.2 Comparison Table

Table 2.1 consolidates the surveyed systems against the four design dimensions this proposal cares about: **M1** what the memory is keyed by, **M2** what quantity is ranked, **M3** whether the cost of a probe enters that quantity, and **M4** whether the evidence is returned to the proposer.

Table 2.1: Representative works against the four dimensions. "—" means the dimension is absent from the system.

#	System	Yr	M1 key	M2 ranked quantity	M3	M4	Headline result
1	GenProg [2,11]	2012	—	— (suite order)			55/105 bugs at $8 each
2	Frtp [5]	2013	failure class	kill *count*			Faster than GenProg on 15/16
3	Cost-cognizant TCP [8]	2001	—	coverage / cost	✓		$APFD_c$ metric
4	Additional greedy [9]	2001	—	marginal coverage			The RTP_{add} family
5	MPP [6]	2020	modif. point	patch killing count			Faster patch validation
6	APR × RTS [4]	2024	—	selection + priority			2.53M patches; sel. > prior.
7	ChatRepair [3]	2024	transcript	— (suite order)		✓	162/337 at $0.42/bug
8	CigaR [15]	2024	transcript	— (suite order)		✓	127K vs 467K tokens/bug
9	Repeated sampling [7]	2025	none	—			23 vs 18 bugs; 65% dupes
10	CegMem (premise)	2026	failure class	— (pool order)		✓	7.07× fewer oracle calls
–	**Caster (this work)**	2026	**test case**	**rate / cost**	✓	✓	See Sec. 4.3

2.3 Research Gap

Three gaps follow from the table, and the third is the one the pilot measures.

Gap 1 — the memory is keyed by a class, so it cannot schedule. Every memory in rows 2, 5, 7, 8 and 10 is indexed by something that is not a test case: a failure class, a modification point, or a conversation transcript. Such an index answers "has this kind of patch been refuted before?". It cannot answer "which probe should run first?", because the probe is not the thing being indexed. CegMem is the clean demonstration: its class index removed 86 % of oracle calls and still left every surviving

call walking the pool in alphabetical order. Re-keying by test case is what makes one record serve the scheduler and the prompt at once.

Gap 2 — the ranked quantity discards the evidence of *not* killing. FRTP and its descendants rank on a count of kills. A count cannot distinguish a case that has failed to refute ten patches from a case that has never been run, because both have count zero, and the distinction is exactly what tells you whether to keep paying for the case. A *rate* — kills over attempts — is the first quantity that reads the denominator, and the first that can decay a case whose early record was luck.

Gap 3 — the cost of a probe is not in the score, and on heterogeneous pools that is the term that matters. Only Elbaum et al.[8] put cost in the score at all, and they assume it is known in advance rather than learned online. This gap is usually invisible because of the unit of account: Lou et al.[4] measure executions, for the reproducibility reason quoted above, and under that unit a cheap probe and an expensive one are the same event. The measured cost distribution of Section 4.3.2 shows what that hides here: a 58.7× spread between the median and the 99th percentile, *inside* the same pool. The pilot's headline (Section 4.3.3) is that the two units disagree in sign — $K/\hat{m}$ pays 23.3 % more executions than FRTP for 15.7 % less CPU — so a study that reports executions alone would conclude FRTP wins.

Two properties of the setting are worth stating, because they explain why Lou et al.'s "selection beats prioritisation" finding does not transfer. First, their subjects are Java programs with method-level test suites, where regression test selection can skip a test because no changed method reaches it. Here a probe is a *whole-program execution on one input*: there is no call graph to cut and nothing to select away, so prioritisation is the only lever available. Second, their cost unit is uniform by construction; here it is not.

2.4 Direct Competitors

Table 2.2: Direct competitors and the differentiator in each case.

Rank	Competitor	Similarity	Our differentiator
#1	FRTP[5]	Same place in the loop; a memory that reorders probes	A rate over attempts, not a count of kills; and divided by a measured cost. Pilot: $-15.7\,\%$ CPU against it
#2	CEGMEM (premise)	Same corpus, same oracle, same proposer	Keyed by test case, not failure class; schedules the probe rather than blocking the patch
#3	MPP[6]	"Patch killing count" table	Same gap as FRTP: a count, no denominator, no cost
#4	Cost-cognizant TCP[8]	Cost appears in the score	Cost is *learned online* from `getrusage` inside the loop, not supplied in advance
#5	Additional greedy[9]	A scheduling policy over the same pool	Measured here as an arm (Sec. 4.3.4) and found no better than pool order on this corpus
#6	APR × RTS[4]	Patch-validation cost, at scale	Selection is unavailable in this setting; and we report CPU, where they report executions
#7	CIGAR[15]	Explicit cost-efficiency goal	Their currency is tokens, ours is oracle CPU; the two are complementary halves of the same bill
#8	Repeated sampling[7]	The strong baseline on the proposer side	It is our RQ3 baseline, not our competitor on RQ1; scheduling is orthogonal to how the draw is made

Chapter 3

Theoretical Background

This chapter sets out the formal objects the proposal manipulates: the repair loop and the two separate costs it incurs, the prioritisation strategies it inherits, the typed memory it starts from, the estimator it ranks by, and the accounting rules that make the measurement meaningful. Implementation specifics are deferred to Chapter 4.

3.1 The Repair Loop and Its Two Costs

Fix a faulty program s_0 and a finite *pool* of test cases $P = \{x_1, \ldots, x_n\}$. Each $x \in P$ carries a reference output, so a candidate program s either agrees with the reference on x or does not; in the latter case x is said to *refute* (or *kill*) s.

Definition 3.1 (Repair episode)**.** An episode with budget B is the sequence, for $t = 1, \ldots, B$: the proposer draws a candidate $s_t \sim \pi(\cdot \mid \mathcal{H}_{t-1})$ from its history $\mathcal{H}_{t-1}$; the oracle probes s_t against cases from P in some order and stops at the first refuting case; if no case refutes s_t the episode *accepts* and stops.

The episode spends two different resources, and conflating them is the error this proposal is organised around.

- **Proposer cost**, paid in model tokens and therefore in money. This is what CIGAR optimises, reporting 127K tokens per bug against a 467K baseline[15].
- **Oracle cost**, paid in CPU seconds executing candidate programs. This is what FRTP[5] and this proposal optimise.

Writing $c(x)$ for the CPU cost of one probe on case x and σ_t for the order the oracle uses in round t, the oracle cost of the episode is

$$C \;=\; \sum_{t=1}^{T} \sum_{j=1}^{r_t} c(\sigma_t(j))\,, \qquad r_t \;=\; \min\{\, j : \sigma_t(j) \text{ refutes } s_t \,\}, \tag{3.1}$$

with $r_t = n$ when nothing refutes. Two facts about Equation (3.1) drive the whole design. First, the order σ_t is free to choose and is the only term the scheduler controls.

Second, minimising the *number* of probes $\sum_t r_t$ is not the same objective as minimising C unless c is constant — and Section 4.3.2 shows it is very far from constant.

Definition 3.2 (Soundness of a schedule)**.** A patch is accepted only after surviving every usable case in P. Acceptance is therefore independent of σ_t: no ordering can accept a patch that another ordering rejects.

Definition 3.2 is what makes a pure cost claim possible. Any difference in repair rate between two scheduling policies is a bug, not a finding, and Section 4.3.6 reports the check.

3.2 Test-Case Prioritisation

Rothermel et al.[9] formalised prioritisation as the search for a permutation of P maximising a rate-of-detection award function, and introduced the two greedy families still in use. *Total* strategies rank x by a static property — usually $|\mathrm{cov}(x)|$, the set of statements it exercises. *Additional* strategies rank by the marginal gain

$$g(x \mid \mathcal{C}) \;=\; \Big|\mathrm{cov}(x) \setminus \mathcal{C}\Big|, \tag{3.2}$$

where $\mathcal{C}$ is what the already-scheduled prefix covers, re-ranking after every pick. The additional family ($\mathrm{RTP}_{\mathrm{add}}$) is the stronger of the two in most empirical comparisons and is the one carried into this study as the `coverage` arm.

Elbaum et al.[8] noted that the standard award function assumes uniform test costs and fault severities, and proposed a cost-cognizant variant. That observation is the ancestor of the denominator in $K/\hat{m}$. The difference is that their cost is an input; here it is a statistic the loop accumulates about itself.

3.3 Fault-Recorded Prioritisation and What It Discards

FRTP[5] ranks each case by its *failing count*: the number of candidate patches it has already falsified. The information is free, because the repair loop produces it as a by-product. Writing $k(x)$ for that count, FRTP schedules in decreasing k.

Two things are missing from k, and naming them precisely is what generates this proposal's score function.

The denominator. Let $n(x)$ be the number of times x has been *run*. A count $k(x)$ without $n(x)$ cannot distinguish a case that was run ten times and refuted nothing from a case that was never run: both have $k = 0$. The ratio $k(x)/n(x)$ is the first quantity that reads the evidence of *not* killing, and it is also the first that lets a case whose early record was luck decay back towards the population mean.

The cost. Nothing in k knows that one probe takes 0.01 s and another 7.5 s. Under a uniform-cost assumption that is harmless. Section 4.3.2 measures the assumption and finds it false by nearly two orders of magnitude, and — worse — finds that cost and kill rate are *positively* correlated within a task, so a policy that maximises kill probability is systematically drawn towards the expensive end of the pool.

3.4 The Premise: CegMem's Typed Memory

The immediate predecessor of this work is CegMem, a counterexample-guided repair loop with a memory of refuted attempts. Its central object is a *failure type* function

$$\theta(s) \;=\; \Big(\; \underbrace{\ell(s_0, s)}_{\text{edit location}} \;,\quad \underbrace{\varphi(s)}_{\text{violated property}} \;\Big), \tag{3.3}$$

where ℓ is computed by diffing the candidate against the *faulty* source it started from — never against the reference — and φ is the exception class, a timeout, or a shape of output divergence. Two granularities ship: a coarse one whose location is always `whole_program`, and a fine one whose location is the changed line range `L`a–`L`b, falling back to a `wholesale` tag when the edit is too large or too scattered to name one place. Because ℓ is a pure diff, it is computable on a fresh candidate *before* any sandbox run, which is what lets the memory act as a guard.

The study ran 309 ConDefects tasks × 5 seeds at budget 20 (1,545 cells per arm) with `gpt-4o-mini`, comparing three memory conditions. Table 3.1 gives the result.

Table 3.1: CegMem main grid: 309 tasks, 5 seeds, budget 20, 1,545 paired cells per arm, `gpt-4o-mini`. Per-episode means.

Arm	Repair rate	Oracle calls	Sandbox runs	Guard (s)	Prompt tokens
no memory	0.5262	11.1094	92.58	0.00	5,365
untyped memory	0.5256	1.4615	38.15	14.50	5,370
typed memory	**0.5812**	**1.5709**	39.47	14.15	7,005

Typed memory beat untyped on repair rate by a wide margin — 144 cells flipped to

success against 58 the other way, $p = 1.3 \times 10^{-9}$ — and both memories cut oracle calls by about 7× against no memory at all. The index behaved as designed: over 23,975 guarded rounds the type-matched bucket was non-empty 59.4 % of the time, and when it was, a stored counterexample still refuted the new candidate 97.6 % of the time.

What that study left open, and why this one exists. Read the columns in the right order and the residual is obvious. The memory removed 86 % of oracle *calls*, but sandbox *runs* fell only from 92.6 to 39.5 — a factor of 2.35, not 7 — and the guard itself consumed 14.15 s per episode. The reason is structural. A class-keyed index decides *whether* to pay the oracle; once the decision is "pay", the oracle still walks the pool in whatever order the benchmark shipped. Nothing in θ is a test case, so nothing in θ can order one.

A second residual is on the proposer side. Typed steering cut the duplicate patch rate only from 0.2653 to 0.2358, and it did so while growing the prompt by a factor of 1.30. The steering was phrased as a prohibition — "this class of edit has been eliminated" — because a class is a category and a category can only be forbidden. That framing is also known to misfire: in CegMem's own measurements a failure class the model would otherwise emit 5.4 % of the time was emitted 14.7 % of the time once the prompt named it, a 2.7× priming inversion. A test case, being an observation, can instead be stated as a requirement to satisfy, which is the hypothesis RQ3 tests.

3.5 The $K/\hat{m}$ Index

Give every case x two online statistics maintained by the loop: $n(x)$, the number of times x has been run, and $k(x)$, the number of those runs that refuted the candidate. Treat the per-case kill probability as a Bernoulli parameter with a Beta conjugate prior, so that the posterior mean is

$$K(x) \;=\; \frac{k(x) + \alpha}{n(x) + \alpha + \beta}. \tag{3.4}$$

Two priors ship. Jeffreys' invariant prior $\mathrm{Beta}(\frac{1}{2}, \frac{1}{2})$[16] is the standard non-informative choice, but it places an unseen case at $K = 0.5$, which outranks a proven refuter whose record has decayed — $k = 2, n = 10$ gives $K = 0.227$ — and under homogeneous costs that degenerates into a full-pool sweep. The default is therefore $\mathrm{Beta}(\frac{1}{2}, 2)$, prior mean 0.2: above this corpus's base rate and below one half. Exploration is handled explicitly by an ε-greedy term rather than smuggled into the prior.

Let $\hat{m}(x)$ be the running mean CPU cost of the probes actually paid on x, with an

unseen case taking the cold-start value defined in Section 4.2.1. The schedule ranks by

$$\text{score}(x) \;=\; \frac{K(x)}{\hat{m}(x)^{\,c}}, \qquad c \geq 0, \tag{3.5}$$

where $c = 1$ recovers the plain ratio $K/\hat{m}$, $c = 0$ recovers a pure kill-rate policy, and intermediate c interpolates. Section 4.3.5 measures the exponent rather than assuming it.

The intuition is a rate of return: $K(x)$ is the probability that spending $\hat{m}(x)$ seconds ends the round, so $K/\hat{m}$ is expected refutations per CPU second, and greedily taking the highest-return probe first is the natural heuristic for Equation (3.1). Two properties keep it honest. The order is built *once* per round from the ledger as it stood when the round began, so nothing is re-ranked mid-round; and only probes actually *paid* update n, while only the refuting probe updates k.

3.6 Cost Accounting

Three rules govern every number reported in this proposal, and each exists because violating it produces a plausible wrong answer rather than an error.

1. **CPU time, not wall clock.** Cost is the delta of `getrusage(RUSAGE_CHILDREN)` across the probe. Wall clock is contaminated by sibling processes, which would make $\hat{m}$ a function of machine load; since the measurement runs several workers in parallel, using wall clock would make the learned cost model an artefact of the harness.

2. **Executions reported alongside, never instead.** The prioritisation literature reports executions for reproducibility[4], so both units are reported for every arm — and Section 4.3.3 shows why that matters.

3. **Timeouts are quarantined.** A contest solution can exceed the clock by being correct but slow, so a timeout is weaker evidence than a wrong answer; timed-out cases are excluded from the cost model rather than entered at the timeout value, which would otherwise dominate $\hat{m}$.

3.7 Benchmark Infrastructure

ConDefects[10] collects real faulty submissions from AtCoder contests together with the same author's accepted version and the contest's own test data: 1,254 Java and 1,625 Python faulty programs. It was built to address data leakage in LLM evaluation,

since its faults post-date the training cut-offs of the models commonly benchmarked on Defects4J[13]. Three of its properties matter here. Every test input ships with the output the judge accepted, so a refutation is a concrete input/output pair rather than a pass/fail bit — which is what makes a counterexample expressible as a positive requirement. A submission carries no problem statement, so every prompt must supply worked examples, identically in every condition. And the reference implementation is visible to the oracle only and never enters a prompt.

Chapter 4

Proposed Framework

4.1 System Overview

CASTER is a repair loop whose memory is a per-*test-case* ledger. Figure 4.1 shows the data path of one round. The proposer draws a candidate; the ledger produces a probe order; the oracle walks that order and stops at the first refutation; the refuting case updates the ledger and becomes the evidence the next prompt carries.

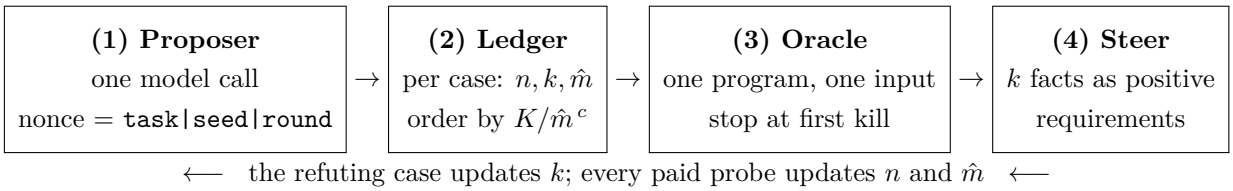


Figure 4.1: One round of the CASTER loop. Stage 2 is the contribution of RQ1–2; stage 4 is the contribution of RQ3–4 and is not exercised by the pilot.

Table 4.1: Per-round data flow.

Stage	Input	Process	Output
S1	history, nonce	one model call; parse candidate program	candidate s_t
S2	ledger $(n, k, \hat{m})$	score each case by Eq. (3.5); freeze the order	permutation σ_t
S3	s_t, σ_t	run s_t on cases in order; stop at first refutation	killer $x^\star$, r_t probes, CPU
S4	$x^\star$, ledger	select k proven cases; render as requirements	round-t+1 prompt

The implementation is a Python package with ten command-line verbs (`freeze`, `propose`, `evaluate`, `audit`, `merge`, `analyse`, `report`, `serve`, `preflight`, `smoke`) and three load-bearing module boundaries, each asserted by a test: `bench/` is the only module that reads a benchmark, so porting touches nothing outside it; `exec/ledger.py`

has no I/O and no benchmark imports, so the whole scheduler is testable without a benchmark, a sandbox or a model; and `steer/` does not import the verifier, so evidence selection reads the ledger and never runs anything.

4.1.1 System Requirements

Functional. The loop shall maintain, per test case, the triple $(n, k, \hat{m})$ and derive a probe order from it once per round **(FR1)**. The order shall be frozen for the duration of a round, so that no case is re-ranked after probing begins **(FR2)**. Probing shall stop at the first refutation, and only paid probes shall update n and $\hat{m}$ while only the refuting probe updates k **(FR3)**. A patch shall be accepted only after surviving every usable case, so that acceptance is independent of the order **(FR4)**. Evidence returned to the proposer shall be rendered as a positive requirement over a concrete input/output pair, never as a named prohibition, and shall never name a case whose content is absent from the prompt **(FR5)**.

Non-functional. Probe cost shall be measured as a `getrusage(RUSAGE_CHILDREN)` delta, never as wall clock **(NFR1)**. Because `getrusage` is process-global, phase-2 parallelism shall be process-per-shard rather than threaded, and the sandbox shall raise rather than let a thread pool mis-attribute a sibling's CPU **(NFR2)**. Round 1 of a steered arm carries no facts and shall therefore be byte-identical to round 1 of an unsteered arm and share its response-cache entry, which requires the draw nonce to be independent of the policy **(NFR3)**. No (patch, probe) pair shall be executed twice under one protocol, and the identity shall be instrumented rather than asserted in prose **(NFR4)**. Every value that changes a measured quantity shall be either in a cache key or in the shard's protocol record, and a merge shall refuse to join across a disagreement **(NFR5)**.

4.2 Module Implementation Details

4.2.1 The Ledger

The ledger is pure statistics over probe names. It stores $n(x)$, $k(x)$ and the accumulated CPU $S(x)$, and exposes $K(x)$ by Equation (3.4) and $\hat{m}(x) = S(x)/n(x)$. Three details are load-bearing.

Cold start. A case never run has no $\hat{m}$. Entering it at zero would make it infinitely attractive; entering it at the pool mean would make the first round's order arbitrary. The ledger uses the maximum of the observed per-case means, which is pessimistic, so

an unmeasured case must earn its place through K rather than through an optimistic cost guess.

Exploration. With probability $\varepsilon = 0.15$ a position in the order is filled uniformly at random from the remaining cases rather than greedily. Without it a task whose first probe refutes everything reaches the prompt having measured exactly one case, and both the cost model and the k-fact evidence contract degenerate.

Freezing. The order is built lazily, one position at a time, from the ledger as it stood when the round began, and construction stops at the kill. The unprobed tail changes neither cost nor the ledger, so this is the same experiment at a fraction of the work — a property the replay study in Section 4.3.4 depends on.

4.2.2 The Oracle and the Stores

One probe is one program on one input under a hard timeout, with CPU read from `getrusage`. A verdict store memoises $(\text{patch}, \text{probe})$ outcomes so that no pair is executed twice within a protocol; timeouts are quarantined rather than reused, because a contest solution near the clock edge is not deterministic. A stored verdict and a fresh one disagreeing halts the run.

4.2.3 The Steering Contract

The prompt is assembled from fixed slots rather than by string concatenation at call sites, and six rules govern it. The two that matter for this proposal: evidence is selected by $K \times$ legibility rather than by $K/\hat{m}$, because the scan pays CPU while the prompt pays tokens and they are different currencies; and the number K that reaches the prompt is the raw rate, because a sentence like "this input refutes 43 % of attempts" is a claim about the world and must stay true. Only *proven* cases may be shown — a case with a prior and no record is a guess, and a wrong guess primes. Inputs longer than 400 characters render as a shape with the first line verbatim.

4.3 Preliminary Validation

4.3.1 Why the Pilot Runs on the Predecessor's Data

The pilot deliberately does *not* run on the CASTER repository. The reason is a property of the data rather than convenience.

A scheduler can only be compared against alternatives if the verdict of *any* case on *any* patch is known, because an alternative order probes cases the deployed order never

reached. A running repair loop does not produce that: it stops at the first refutation, so its log is censored past that point, and a counterfactual order cannot be replayed from it. The CegMem study, however, ran every stored patch against its task's whole pool in order to measure redundancy, and therefore left behind a *complete* patch × case verdict matrix. That matrix is what makes an exact counterfactual replay possible, and it exists for the predecessor's data and nowhere else.

The pilot therefore does two things on that data. First, it *replays* eleven scheduling policies exactly, reconstructing what each would have probed and paid. Second — because a replay is only as good as its cost model — it *re-executes* the loop for real under three of those policies and measures the CPU with `getrusage`, so the headline claim rests on measurement rather than on simulation.

4.3.2 The Arena

The CegMem corpus holds 309 ConDefects Python tasks whose pools have a median of 34 cases (mean 38.9, range 15–144). The pilot's arena is the subset for which three independent measurements all exist: the complete verdict matrix, a measured per-case CPU cost of the *reference* program, and a per-case line-coverage trace of that program (needed only by the coverage arm). Intersecting them, and dropping cases that timed out, leaves **68 tasks, 2,523 test cases and 4,670 stored patches**.

Table 4.2: The pilot arena, and the two distributions that motivate the design.

Quantity	Value	Reading
Tasks / cases / patches	68 / 2,523 / 4,670	The replay universe
Per-case reference CPU, median	0.0763 s	Cost is *not* uniform: the 99th
90th percentile	1.740 s	percentile is 58.7× the median,
99th percentile	4.476 s	and the spread is *within* pools,
median within-task CV	0.571	not between tasks
Per-case kill rate, mean	0.463	Kill ability is spread out, not
median	0.306	concentrated in a handful of
cases that never kill	16.8 %	"super" cases
median within-task $\mathrm{corr}(\log c, K)$	**+0.172**	**Expensive cases are *more* likely to kill, so a kill-rate-only policy is actively drawn towards the expensive end of the pool**

The last row is the one that makes the proposal non-obvious. If cost and kill rate were independent, dividing by cost would be a free improvement of second order. They are positively correlated, so a policy maximising K alone systematically buys the expensive probes — and the division by $\hat{m}$ is not a refinement but a correction.

4.3.3 Measured Head-to-Head

Three policies were re-executed for real on the arena: `alphabet` (the pool order the benchmark ships, i.e. the incumbent), `fault-record` (FRTP as published, ranking on the kill count), and $K/\hat{m}$. Each ran 68 tasks $\times$ 5 seeds $\times$ 20 rounds, giving 6,800 measured rounds per policy and 20,400 in total. Table 4.3 and Figure 4.2 give the result; confidence intervals are from a cluster bootstrap over tasks (10,000 resamples), and p-values from a Wilcoxon signed-rank test over per-task totals.

Table 4.3: Measured head-to-head on the arena. 6,800 rounds per policy, CPU from `getrusage`. "cheaper on" counts tasks where the row is cheaper than $K/\hat{m}$.

Policy	Oracle CPU (s)	probes / round	CPU / round (s)	**CPU / probe (s)**
`alphabet` (incumbent)	14,842.8	6.493	2.1828	0.3362
`fault-record` (FRTP)	4,189.8	**1.641**	0.6162	0.3754
$K/\hat{m}$ **(ours)**	**3,531.9**	2.024	**0.5194**	**0.2567**

Table 4.4: Paired contrasts, cluster bootstrap over 68 tasks.

Contrast	Δ CPU	95 % CI	p	tasks won	Δ probes
$K/\hat{m}$ vs `alphabet`	$-76.20\,\%$	$[-84.94, -61.17]$	1.7×10^{-11}	63 / 68	$-68.84\,\%$
$K/\hat{m}$ vs `fault-record`	$-15.70\,\%$	$[-28.93, -5.06]$	0.011	42 / 68	$\mathbf{+23.30}\,\%$
`fault-record` vs `alphabet`	$-71.77\,\%$	$[-82.91, -50.69]$	4.2×10^{-9}	62 / 68	$-74.73\,\%$

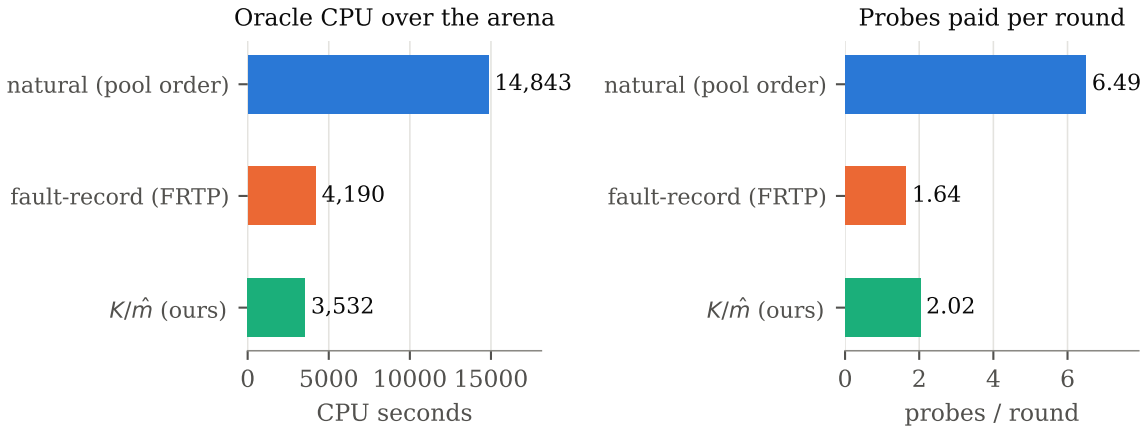


Figure 4.2: Measured oracle cost and probe count on the arena. The two panels rank the policies in *opposite* orders.

Reading. Against the incumbent the result is large and uncontroversial: $K/\hat{m}$ removes 76.2 % of the oracle's CPU and is cheaper on 63 of 68 tasks. The interesting comparison is against FRTP, and it is interesting because the two units of account **disagree in sign**. $K/\hat{m}$ pays 23.3 % *more* probes and 15.7 % *less* CPU. The right-hand column of Table 4.3 shows the mechanism directly: FRTP pays 0.3754 s per probe — *more* than the incumbent's 0.3362 s, because ranking on the kill count steers it towards exactly the expensive cases that Table 4.2's last row predicts — while $K/\hat{m}$ pays 0.2567 s, 32 % less per probe. $K/\hat{m}$ does not find refutations in fewer steps; it finds them in cheaper steps.

This is the pilot's methodological contribution, and it is a caution rather than a boast. Lou et al. [4] report executions rather than seconds for a defensible reproducibility reason, and on a uniform-cost pool that reporting choice is free. On this pool it would have inverted the conclusion: measured in executions, FRTP wins by 23 %.

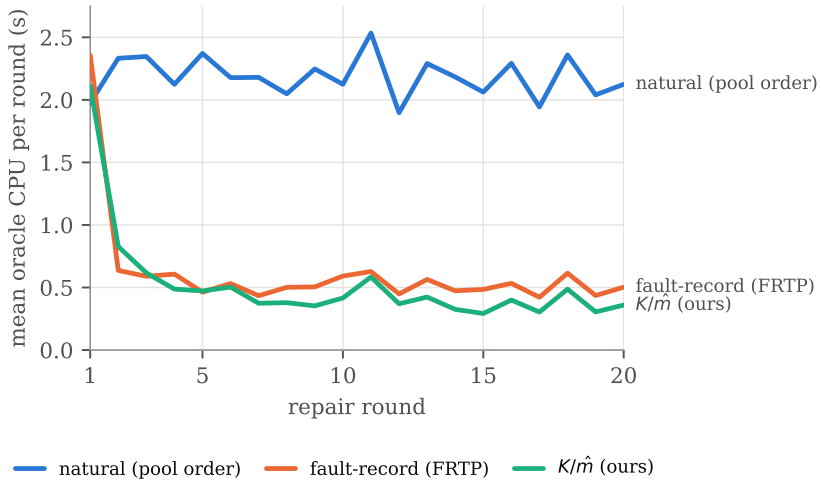


Figure 4.3: Measured oracle CPU per round. The incumbent never learns; both memories collapse within about three rounds; $K/\hat{m}$ continues to separate from FRTP as its cost model fills in.

Figure 4.3 shows that the ledger is doing what it is supposed to. In round 1 every policy is identical by construction — the ledger is empty, so there is nothing to rank by — and all three pay about 2 s. By round 5 both memory policies have collapsed to roughly a quarter of that. The two then separate slowly: FRTP plateaus around 0.50 s while $K/\hat{m}$ continues down to 0.359 s by round 20, because its cost estimates keep improving while FRTP has nothing left to learn.

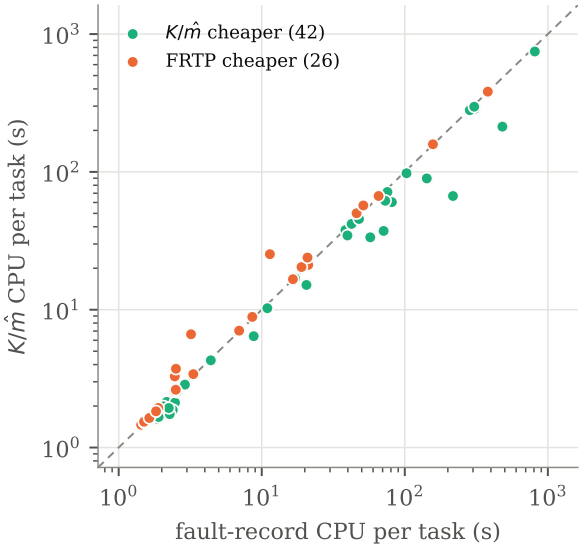


Figure 4.4: Per-task measured CPU, $K/\hat{m}$ against FRTP. The win is broad rather than driven by outliers, but it is not universal.

Figure 4.4 is the honest view of the FRTP comparison. $K/\hat{m}$ is cheaper on 42 of 68

tasks — a clear majority, not a sweep — and the aggregate margin comes partly from the expensive tasks on the upper right, where the cost spread inside the pool is largest. On cheap, homogeneous pools the two policies are interchangeable, which is what the theory predicts.

4.3.4 Replay Head-to-Head

The replay study puts eight further policies on the same arena without paying to execute them, by reconstructing from the complete verdict matrix exactly what each would have probed. It uses 20 seeds per task. Because the replay reuses the measured per-case costs, it is a counterfactual over *orderings*, not a model of execution.

Table 4.5: Replay over the arena, 20 seeds. Δ is against plain $K/\hat{m}$; CIs are a cluster bootstrap over the 68 tasks.

Policy	CPU (s)	probes/rd	Δ vs $K/\hat{m}$	95 % CI	sig.
`natural` (pool order)	18,297.1	6.375	+589.4 %	[+216.8, +990.7]	⋆
`random` (control)	13,275.8	4.252	+400.2 %	[+232.6, +549.1]	⋆
`coverage` ($\mathrm{RTP}_{\mathrm{add}}$)	16,611.3	5.368	+525.8 %	[+192.1, +877.0]	⋆
`cost-only` ($1/\hat{m}$)	8,810.6	5.083	+231.9 %	[+ 78.9, +403.1]	⋆
`kill-only` (K)	2,800.9	1.305	+ 5.5 %	[− 7.3, + 23.6]	
`km` ($K/\hat{m}$)	2,654.2	1.943	0.0 %	—	
$K/\hat{m}^{0.5}$ **(proposed)**	**2,409.4**	1.474	**− 9.2 %**	[− 15.0, − 1.6]	⋆
$K^{0.5}g^{0.25}/\hat{m}^{0.25}$	2,366.4	1.500	− 10.8 %	[− 16.1, − 4.2]	⋆

Three readings follow.

The memory term carries most of the effect. Going from `natural` to `kill-only` takes CPU from 18,297 to 2,801. That is the step that matters most, and it is the step FRTP already takes — which is why FRTP is the right baseline and `natural` is not.

Cost alone is not enough. `cost-only` ranks by $1/\hat{m}$ with no kill information and lands at 8,811: better than pool order, far worse than any policy that knows which cases refute. The two terms are not substitutes.

Additional-greedy coverage does not transfer. The $\mathrm{RTP}_{\mathrm{add}}$ arm reaches 16,611, barely better than pool order and much worse than the random control. Coverage of the *reference* program is nearly constant across a contest task's pool — the median within-task coefficient of variation of covered-line count is 0.073 — so there is almost no marginal coverage to be greedy about. This is a negative result worth stating plainly: the strongest classical prioritiser is useless in this setting.

4.3.5 How Much to Divide by Cost

Equation (3.5) leaves the exponent c free, and the two endpoints are the two published positions: $c = 0$ is the FRTP family's kill-only ranking and $c = 1$ is full cost-cognizance. The sweep measures it.

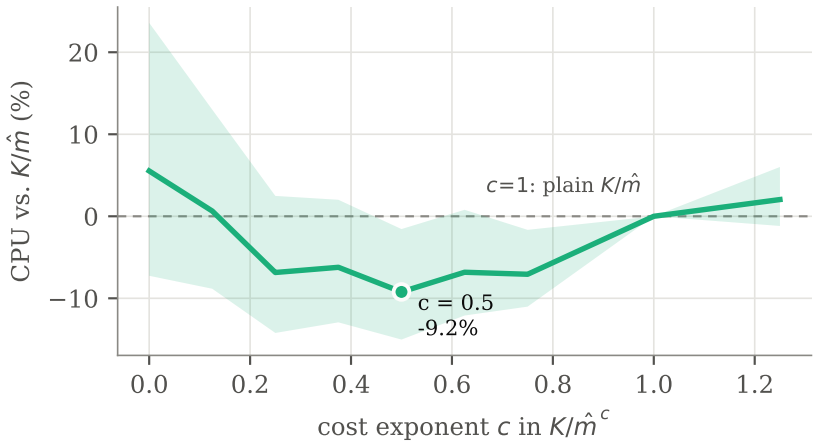


Figure 4.5: Cost-exponent sweep, 20 seeds, paired per task. The optimum is interior: dividing by cost helps, dividing by the *full* cost over-corrects.

The curve is U-shaped with a minimum at $c = 0.5$, which beats plain $K/\hat{m}$ by 9.2 % (CI $[-15.0, -1.6]$). Both endpoints are worse: $c = 0$ costs +5.5 % and $c = 1.25$ costs +2.0 %, neither significantly different from $K/\hat{m}$. The interpretation is that $\hat{m}$ is a noisy online estimate, and a square root is the standard way to use a noisy denominator without letting one cheap-looking early measurement dominate the ranking. The practical conclusion for Phase 2 is that c must be a tuned parameter with a declared default, not a constant fixed at 1 by the name of the method.

A companion sweep added the marginal-coverage term g of Equation (3.2) as a third factor at fifteen (a, b, c) combinations. Of the fifteen, twelve have confidence intervals straddling zero; the largest effect at the operating point a=1, c=0.5 is −2.2 % with CI $[-5.0, +0.9]$. Coverage is therefore **dropped from the proposed score**: it costs a tracing pass per case and buys nothing measurable.

4.3.6 Soundness Check

Definition 3.2 says no ordering can change which patches are accepted. The measured runs confirm it: all three policies accepted exactly 6 patches over the 6,800 rounds each. A discrepancy here would have been a soundness bug rather than a finding, and the check is cheap enough to repeat on every cell in Phase 2.

4.3.7 Reproducibility

Every number in this section is regenerated from three input files — the verdict matrix, the per-case cost table and the per-case coverage table — by the scripts shipped under `report/scripts/`. The replay head-to-head and the exponent sweep were re-run from scratch for this document and reproduce the stored values to the printed precision; the measured block is a direct aggregation of the 33,000-row round log. Generated artefacts are under `report/data/`.

4.4 Discussion

What the evidence supports. Three claims are supported at the pilot's scale. The oracle's cost in a counterexample-guided repair loop is dominated by probe ordering, and a per-case memory removes about three quarters of it. A cost-aware ranking beats the published kill-count ranking by a further 15.7 % of CPU, with the margin coming from cheaper probes rather than fewer ones. And the unit of account is not neutral on this corpus: CPU and executions order the policies oppositely, so a prioritisation result reported in executions alone does not transfer to a deployment that pays seconds.

What it does not support. The pilot says nothing about the proposer. RQ3 and RQ4 — whether a counterexample keyed to a test case and phrased as a positive requirement renormalises the patch distribution, and at what disclosure that turns into fitting the shown tests — are untouched, because the CegMem data was collected under a prohibition-style prompt and no replay can manufacture a counterfactual model call. It also says nothing about scale: 68 tasks is a pilot, the arena is restricted to tasks whose cost and coverage passes finished, and that restriction is not random with respect to task cost. Finally, the 15.7 % margin over Frtp has a wide interval ($[-28.9, -5.1]$) and holds on 42 of 68 tasks; it is a real effect at this scale, not a settled magnitude.

Threats carried into Phase 2. Three are worth naming now. The arena's selection is cost-correlated, so the full corpus may show a smaller margin. The replay reuses one cost measurement per case rather than sampling the cost distribution, which understates variance — the measured block exists precisely so the headline does not depend on the replay. And `gpt-4o-mini` produced every patch in the matrix, so the patch population, and therefore the kill structure the ledger learns, is one model's.

Chapter 5

Evaluation Plan

Nothing in this chapter is a result. Every figure below is a target or a protocol; the only measured evidence in this document is Section 4.3.

5.1 Metrics

The study does not collapse into a single figure of merit, because the two halves of the loop spend different resources and a composite would let one pay for the other. Metrics are grouped by the resource they account for.

Table 5.1: Oracle-side metrics (RQ1, RQ2).

Metric	Definition	Success criterion
CPU seconds / episode	`getrusage` delta summed over paid probes; paired per cell under common random numbers	$K/\hat{m}$ below `natural`, `fault-record` and `coverage`, beyond the cluster bootstrap's interval
`test_calls` / episode	Probe count, reported alongside CPU because the prioritisation literature reports executions	Reported for every arm; *not* a success criterion on its own
CPU per probe	CPU ÷ probes; isolates "cheaper" from "fewer"	Diagnostic; the pilot's mechanism claim lives here
Stepwise decomposition	Paired Δ CPU for each one-variable step of the ablation ladder	At least one step clears the interval

Table 5.2: Proposer-side metrics (RQ3, RQ4).

Metric	Definition	Success criterion
`duplicate_rate`	1 − distinct/total patches in an episode. RQ3's primary	Falls against `no-mem` (independent repeated sampling)
`success@B`	Share of cells repaired within budget B	Rises, or at minimum does not fall, where `duplicate_rate` falls
`first_accept_round`	Round of first acceptance	Reported paired
Prompt / completion tokens, USD	The proposer's bill, since the repair literature prices in tokens[15]	Reported for every Block-B arm
Disclosure leakage	Of refuted patches, the share whose refuting set is *disjoint* from what the prompt showed	Falls monotonically as disclosure rises, or RQ4's axis does not exist

Guaranteed, not measured. `success@B` and `first_accept_round` are identical across every replay arm on every cell, by Definition 3.2. RQ1 is a pure cost claim; a gap there is a soundness bug, and the grid asserts it rather than reporting it.

5.2 The Arm Roster

Twenty-two arms, in two blocks that differ in what they cost to run.

Block A — replay arms, zero model calls (13). Every arm here differs only in the probe order, so the patches are drawn once and reused, and a new arm costs nothing but CPU: `full-suite`, `no-mem`, `natural`, `random`, `fault-record`, `cost-only`, `kill-only`, `coverage`, `km`, `km-cs-median`, `km-cs-q75`, `km-bucket`, `km-explore0`.

Block B — steered arms, one model-call grid each (9). For these the round-t prompt is a function of the ledger at round t, so the draw and the schedule are not separable: `km+steer`, `km+steer-pass`, `km+exclusion`, `km+chat`, `km+steer-k1`, `km+steer-k3`, `km+steer-sample`, `km+steer-noreq`, `km+steer+vs`.

Two framing points govern how the roster is read, and both are consequences of Chapter 2.

- **`natural` is the incumbent, not a neutral control.** Sorting by filename is not a scheduling decision anyone made; it is what any implementation gets for free. It is also not arbitrary — AtCoder's naming puts the small, public, highly discriminating

cases first — so it is *better* than chance, which is exactly why the pilot's random control is cheaper than it. `random` is the control.

- **`no-mem` is the RQ3 baseline, and it is a strong one.** It redraws independently every round under a policy-free nonce, which is precisely independent repeated prompting — the strategy Chen et al.[7] measured at 23 bugs repaired against conversational repair's 18. Block B must beat `no-mem`; beating `natural` proves nothing on the proposer side.

5.3 Ablation Ladder

An ablation chain must change one thing per step or it cannot attribute. `fault-record` → `km` changes two — a count becomes a rate, *and* the rate is divided by cost — so the ladder is built to separate them.

Table 5.3: The ablation ladder. One variable per step.

Step	What changes	What it would mean if it does not clear the interval
`natural` → `cost-only`	cost enters, no kill info	Cost alone is not a usable signal (pilot: it is not, on its own)
`natural` → `kill-only`	kill rate enters, no cost	The memory term is ornament
`natural` → `fault-record`	the published baseline's step	FRTP does not reproduce
`fault-record` → `kill-only`	a **count** becomes a **rate**	Reading the denominator buys nothing
`kill-only` → `km`	the rate is divided by cost	Cost-awareness is ornament
`km` → $K/\hat{m}^{\,c}$	the exponent is tuned	c need not be a parameter (pilot: it does)

5.4 Sensitivity

Four arms ask whether RQ1's answer survives the project's own design choices, and one asks the same of RQ3. They belong in threats to validity, not in the question count: `km-cs-median` and `km-cs-q75` vary the cold-start rule for $\hat{m}$; `km-bucket` replaces the continuous score with buckets; `km-explore0` removes exploration — and is additionally a *precondition* of RQ4, because without exploration a task whose first probe refutes

everything reaches the prompt with one fact, k=2 degenerates to k=1, and the k sweep compares k=1 against itself; `km+steer+vs` diversifies the *draw* rather than the evidence.

5.5 Protocol and Tooling

Table 5.4: Tools and their role.

Tool	Role
ConDefects[10]	Fault corpus, test pools and reference outputs
Python `subprocess` + `resource`	The sandbox: one program, one input, hard timeout, CPU via `getrusage`
OpenAI-compatible endpoint	The proposer; one model per grid, with a cross-run response cache
`caster` CLI (ten verbs)	Corpus freeze, phase-1 draw, phase-2 grid walk, full-scan audit, shard merge, analysis, HTML report
Cluster bootstrap + Wilcoxon	Interval estimation over tasks; paired tests over per-task totals

Grid. The corpus gate admits every sound fault rather than a screened subset, producing on the order of 1,244 faults; banding by pool size and contest era happens in analysis, after treatment. Cells are (task × seed × policy), written atomically and resumably, with the done-index written after the rows. A merge runs six audits before anything is treated as evidence.

Common random numbers. Round 1 carries no facts, so a steered prompt must be byte-identical to an unsteered one and share its cache entry. That works only if the draw nonce is `task|seed|round` and is *not* a function of the policy. Deriving it from the cell key — which does contain the policy — would kill CRN silently while buying every completion once per arm.

5.6 Baseline Comparisons

Table 5.5: Tier A — reproduced head-to-head on our corpus.

System	Role	Expected outcome
`natural`	The incumbent order	The $K/\hat{m}$ substantially cheaper (pilot: $-76\,\%$)
FRTP[5]	The published repair-loop prioritiser, ranking on the kill count	The $K/\hat{m}$ cheaper in CPU, *more expensive* in executions (pilot: $-15.7\,\%$ / $+23.3\,\%$)
RTP_{add}[9]	The strongest classical prioritiser	No better than pool order here (pilot: confirmed)
`no-mem`	Independent repeated sampling[7]	The RQ3 baseline Block B must beat
`km+chat`	A CHATREPAIR-style transformation	Higher duplicate rate and faster prompt growth than the requirement contract

Table 5.6: Tier B — cited qualitatively from the original papers; not reproducible within this project's budget.

System	Reported result	Relation to this work
APR × RTS[4]	2.53M patches; selection beats prioritisation in Java	Different setting: no call graph to select on; and they report executions, we report CPU
CigaR[15]	127K vs 467K tokens / bug	Complementary currency: they cut the proposer's bill, we cut the oracle's
ChatRepair[3]	162/337 bugs at $0.42 each	The conversational incumbent our Block-B contract is measured against
MPP[6]	Modification-point-aware patch killing count	Same missing denominator and missing cost as Frtp

5.7 Threats to Validity

Construct. Cost is CPU, not wall clock or money. A deployment that pays for wall clock on a loaded machine, or for a managed test service priced per invocation, would weight the two units differently — and the pilot shows those weightings disagree. Both are therefore reported for every arm.

Internal. The ledger learns from the patches the proposer happens to produce, so policy and data are coupled within an episode. Common random numbers hold round 1 identical across arms, and the full-scan `audit` pass establishes the kill matrix independently of any arm, so no arm's ordering can contaminate the cost table.

External. One benchmark, one language, one proposer model, and a competitive-programming fault distribution with small single-file programs and whole-program probes. Transfer to Java projects with method-level suites is not claimed; the setting there is the one where Lou et al.[4] find selection to dominate.

Conclusion. Per-task totals are not independent across seeds, so intervals come from a cluster bootstrap over tasks rather than over cells, and paired tests run on per-task totals.

Chapter 6

Summary and Future Plan

6.1 Summary

A counterexample-guided repair loop spends a counterexample twice and, in the usual implementation, wastes it both times: once by running the oracle in an order nobody chose, and once by letting the proposer redraw from the mode it has just drawn from. This proposal addresses the first waste and designs for the second.

The route to it ran through a previous study. CegMem gave the repair loop a memory keyed by a failure type — edit location crossed with violated property — and that memory worked: oracle calls per episode fell from 11.11 to 1.57, and the repair rate rose from 0.526 to 0.581 over 309 tasks. Reading its columns carefully is what produced the present question. Oracle *calls* fell 7.07×, but sandbox *runs* fell only 2.35×, because a class-keyed index decides *whether* to pay the oracle and has nothing to say about *how* the oracle spends once it is paid. A failure class is a category; a test case is an observation; only the latter can be ordered, costed, or phrased to a model as a requirement.

Caster therefore re-keys the memory by test case and ranks probes by $K/\hat{m}^{\,c}$ — an online Beta-posterior kill rate over an online mean CPU cost. The preliminary study of Section 4.3, run on the frozen CegMem data because only that data carries the complete patch × case verdict matrix an exact counterfactual replay needs, establishes the following on 68 tasks, 2,523 cases and 4,670 patches:

- **Against the incumbent**, measured over 6,800 rounds per policy, $K/\hat{m}$ removes 76.2 % of oracle CPU (95 % CI $[-84.9, -61.2]$, $p = 1.7 \times 10^{-11}$), and is cheaper on 63 of 68 tasks.
- **Against the published Frtp baseline**, $K/\hat{m}$ removes a further 15.7 % of CPU (CI $[-28.9, -5.1]$, $p = 0.011$, 42 of 68 tasks) *while paying* 23.3 % *more probes*. The mechanism is visible in the cost per probe: Frtp pays 0.375 s, which is *more* than the incumbent's 0.336 s, because ranking on kill count steers it towards expensive cases; $K/\hat{m}$ pays 0.257 s.

- **The unit of account is not neutral.** CPU and executions rank the two policies oppositely on this corpus, so a prioritisation result reported in executions — as the literature reports it, for good reproducibility reasons — would have inverted the conclusion.
- **The exponent is a parameter, not a constant.** The cost-exponent sweep has an interior optimum at $c = 0.5$, beating plain $K/\hat{m}$ by 9.2 % (CI $[-15.0, -1.6]$); both c=0 and c=1.25 are worse.
- **A negative result.** Additional-greedy coverage, the strongest classical prioritiser, performs barely better than pool order and worse than random here, because reference coverage is nearly constant across a contest task's pool (median within-task CV 0.073). It is dropped from the score.
- **Soundness holds.** All three measured policies accepted exactly the same number of patches, as Definition 3.2 requires.

What the pilot does not establish is equally important. It says nothing about the proposer: RQ3 and RQ4 are untouched, because no replay can manufacture a counterfactual model call. It is a pilot at 68 tasks, drawn from a subset whose selection correlates with cost. And every patch in the matrix came from one model.

6.2 Future Direction

The remaining work is organised so that the pipeline is frozen before the evaluation runs, rather than tuned while it runs. **Every item below is Phase 2 work and none of it has been executed.**

- **Run the Block-A grid on the full corpus.** Thirteen replay arms over roughly 1,244 gated faults, with the `audit` full-scan pass establishing the kill matrix and the cost table independently of any arm. This re-establishes RQ1 at scale and runs the ablation ladder of Section 5.3 for RQ2.
- **Fix c by protocol, not by hindsight.** The pilot's $c = 0.5$ was selected on the pilot arena. Phase 2 declares c before the corpus run and reports the sweep as sensitivity, so the headline is not a selected maximum.
- **Implement and run Block B.** Nine steered arms, each a full model-call grid. This is the first test of the proposal's second half — that a counterexample keyed to a test case and phrased as a positive requirement moves the proposal distribution where

a prohibition and a transcript do not — measured against independent repeated sampling, which is the published stronger strategy[7].

- **Measure the disclosure dose (RQ4).** The k sweep and the disclosure sweep are one dose-response curve; `km-explore0` must be in place first or the curve compares k=1 against itself.
- **Report both bills together.** The oracle's CPU and the proposer's tokens are halves of one cost. CIGAR optimised the second and reported 127K against 467K tokens per bug[15]; this project optimises the first and should report the pair rather than either alone.
- **Beyond this setting (thesis horizon).** Whole-program probes and single-file faults are what make selection unavailable and prioritisation the only lever. Carrying the cost-aware ledger into a setting with method-level suites, where selection *is* available[4], is the natural generalisation and is out of scope here.

6.3 Limitations

Table 6.1: Limitations and planned mitigations.

Limitation	Impact	Mitigation
The CASTER grid has not been run; evidence is a pilot on the predecessor's data	RQ1 is established at 68 tasks, not at corpus scale; RQ2 only partially	Phase 2 Block-A grid on ≈ 1,244 faults with the full ablation ladder
Arena selection correlates with cost (only tasks whose cost and coverage passes finished)	The 15.7 % margin over FRTP may shrink	The full corpus admits every sound fault; no pre-treatment screen
One proposer model (`gpt-4o-mini`) produced every patch in the matrix	The kill structure the ledger learns is one model's	A second proposer in Phase 2 sensitivity
The replay reuses one cost sample per case	Understates cost variance	The headline rests on the re-executed block, not the replay
$c = 0.5$ was selected on the same arena it is reported on	A selected maximum is optimistically biased	Declare c before the corpus run; report the sweep as sensitivity
Whole-program, single-file, stdout-compared faults	No claim transfers to project-scale Java repair	Stated as external threat; selection-capable settings are future work
CPU is the cost; money and wall clock are not	A differently priced deployment weights the units differently	Both CPU and executions reported for every arm

6.4 Timeline

The project runs across the 2026 calendar year in two phases separated by the institutional summer break. Phase 1 covers the work reported here: the CEGMEM study that generated the question, the literature consolidation, the CASTER implementation and its unit tests, and the pilot. Phase 2 covers the grid, the steered block, and the writing.

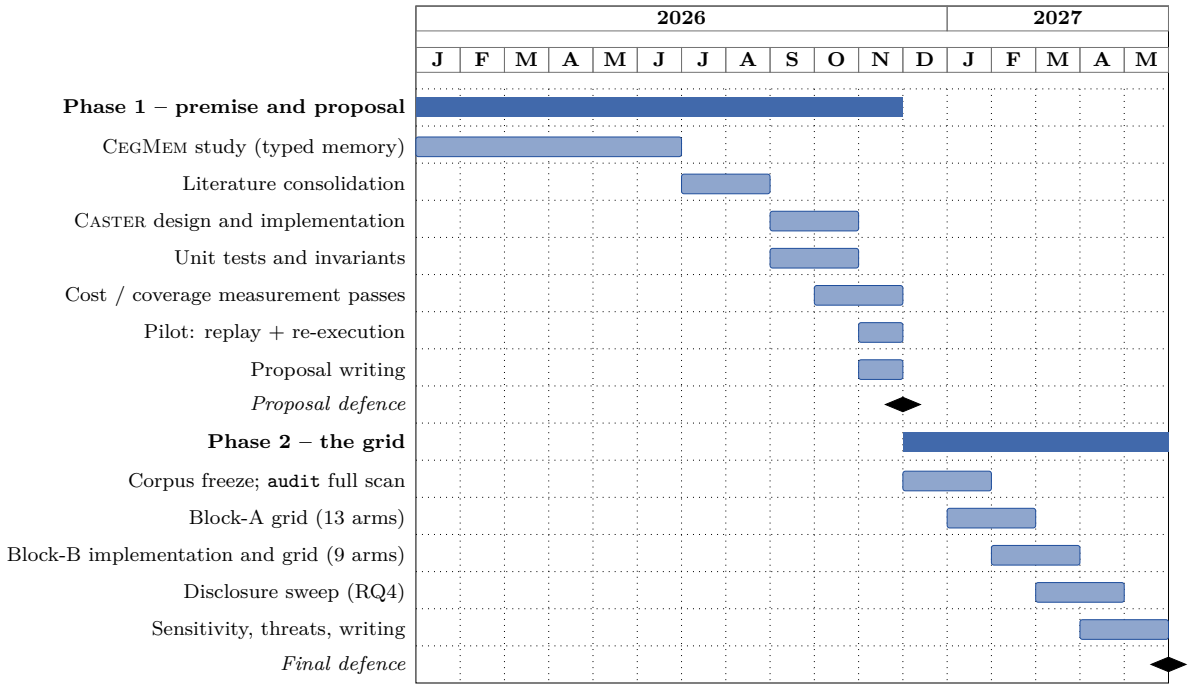


Figure 6.1: Project timeline. Everything up to and including the proposal-defence milestone has been carried out; every bar below it is planned work. Deliverables per work package are in Table 6.2.

Table 6.2: Work packages by phase.

Phase	Period	Main focus	Deliverable
	Jan–Jun	CegMem: typed memory, the 309-task grid	The premise, and the residual that generated RQ1
Phase 1	Jul–Aug	Literature consolidation	Related-work matrix; the gap statement of Sec. 2.3
	Sep–Oct	Caster design and implementation	Ten verbs; the three module boundaries; 263 unit tests
	Oct–Nov	Cost and coverage measurement passes	`cost_all`, `cov_all`, the verdict matrix
	Nov	Pilot and proposal	Sec. 4.3; this document; defence
	Dec–Jan	Corpus freeze; `audit` full scan	Kill matrix and frozen cost table, arm-independent
Phase 2	Jan–Feb	Block-A grid, 13 arms	RQ1 at corpus scale; the RQ2 ablation ladder
	Feb–Mar	Block-B implementation and grid, 9 arms	RQ3: duplicate rate against repeated sampling
	Mar–Apr	Disclosure sweep	RQ4 dose-response curve
	Apr–May	Sensitivity, threats, writing	Final manuscript and defence

Two buffers are built into the plan. The evaluation protocol — the corpus gate, the pool order, the cost table and the declared value of c — is frozen at the end of Phase 1, so unexpected behaviour in Phase 2 can be analysed rather than hidden by continuous retuning. And because Block A costs no model calls, the thirteen replay arms can be re-run freely whenever the analysis changes, which is what makes the ablation ladder affordable at all.

References

[1] Claire Le Goues, Michael Pradel, and Abhik Roychoudhury. Automated program repair. *Communications of the ACM*, 62(12):56–65, 2019. doi: 10.1145/3318162.

[2] Claire Le Goues, ThanhVu Nguyen, Stephanie Forrest, and Westley Weimer. GenProg: A generic method for automatic software repair. *IEEE Transactions on Software Engineering*, 38(1):54–72, 2012. doi: 10.1109/TSE.2011.104.

[3] Chunqiu Steven Xia and Lingming Zhang. Automated program repair via conversation: Fixing 162 out of 337 bugs for $0.42 each using ChatGPT. In *Proceedings of the 33rd ACM SIGSOFT International Symposium on Software Testing and Analysis (ISSTA)*, pages 819–831, 2024. doi: 10.1145/3650212.3680323.

[4] Yiling Lou, Jun Yang, Samuel Benton, Dan Hao, Lin Tan, Zhenpeng Chen, Lu Zhang, and Lingming Zhang. When automated program repair meets regression testing—an extensive study on two million patches. *ACM Transactions on Software Engineering and Methodology*, 33(7):Article 180, 2024. doi: 10.1145/3672450.

[5] Yuhua Qi, Xiaoguang Mao, and Yan Lei. Efficient automated program repair through fault-recorded testing prioritization. In *Proceedings of the 29th IEEE International Conference on Software Maintenance (ICSM)*, pages 180–189, 2013. doi: 10.1109/ICSM.2013.29.

[6] Yazhini Venugopal, Phung Quang-Ngoc, and Lee Eunseok. Modification point aware test prioritization and sampling to improve patch validation in automatic program repair. *Applied Sciences*, 10(5):1593, 2020. doi: 10.3390/app10051593.

[7] Aolin Chen, Haojun Wu, Qi Xin, Steven P. Reiss, and Jifeng Xuan. Studying and understanding the effectiveness and failures of conversational LLM-based repair. *arXiv preprint arXiv:2503.15050*, 2025.

[8] Sebastian Elbaum, Alexey Malishevsky, and Gregg Rothermel. Incorporating varying test costs and fault severities into test case prioritization. In *Proceedings of the 23rd International Conference on Software Engineering (ICSE)*, pages 329–338, 2001. doi: 10.1109/ICSE.2001.919106.

[9] Gregg Rothermel, Roland H. Untch, Chengyun Chu, and Mary Jean Harrold. Prioritizing test cases for regression testing. *IEEE Transactions on Software Engineering*, 27(10):929–948, 2001. doi: 10.1109/32.962562.

[10] Yonghao Wu, Zheng Li, Jie M. Zhang, and Yong Liu. ConDefects: A complementary dataset to address the data leakage concern for LLM-based fault localization and program repair. In *Companion Proceedings of the 32nd ACM International Conference on the Foundations of Software Engineering (FSE)*, pages 642–646, 2024. doi: 10.1145/3663529.3663815.

[11] Claire Le Goues, Michael Dewey-Vogt, Stephanie Forrest, and Westley Weimer. A systematic study of automated program repair: Fixing 55 out of 105 bugs for $8 each. In *Proceedings of the 34th International Conference on Software Engineering (ICSE)*, pages 3–13, 2012. doi: 10.1109/ICSE.2012.6227211.

[12] Zichao Qi, Fan Long, Sara Achour, and Martin Rinard. An analysis of patch plausibility and correctness for generate-and-validate patch generation systems. In *Proceedings of the 24th ACM SIGSOFT International Symposium on Software Testing and Analysis (ISSTA)*, pages 24–36, 2015. doi: 10.1145/2771783.2771791.

[13] René Just, Darioush Jalali, and Michael D. Ernst. Defects4J: A database of existing faults to enable controlled testing studies for Java programs. In *Proceedings of the 23rd International Symposium on Software Testing and Analysis (ISSTA)*, pages 437–440, 2014. doi: 10.1145/2610384.2628055.

[14] Yiling Lou, Junjie Chen, Lingming Zhang, and Dan Hao. A survey on regression test-case prioritization. In Atif Memon, editor, *Advances in Computers*, volume 113, chapter 1. Academic Press, 2019. ISBN 9780128160718.

[15] Dávid Hidvégi, Khashayar Etemadi, Sofia Bobadilla, and Martin Monperrus. CigaR: Cost-efficient program repair with LLMs. *arXiv preprint arXiv:2402.06598*, 2024.

[16] Harold Jeffreys. An invariant form for the prior probability in estimation problems. *Proceedings of the Royal Society of London. Series A*, 186(1007):453–461, 1946. doi: 10.1098/rspa.1946.0056.